

# NRStitcher Preprocessing Pipeline: Project Report

---

**Author:** Allison Cairns **Date:** February 2026 **Environment:** Python 3.9 (Conda `stitch_app`), Windows OS (Misha Cluster / Local workstation targets)

---

## Executive Summary

This project involved the development of a comprehensive preprocessing and visualization pipeline (`app.py` and `core.py`) for the `pi2` (NRStitcher) software stack. The overarching goal was to streamline the generation of High-Performance Computing (Slurm) and local batch scripts, validate 100GB+ microscopy datasets before processing, and introduce robust Quality Control (QC) analytics for non-linear warping and intensity drift.

The application was built as a Streamlit Graphical User Interface (GUI), focusing on usability, defensibility, and reproducibility, effectively replacing error-prone manual scripting.

## Major Implementations & Timeline

### Initial Foundation (Mid-February 2026)

- **Core Parsing Engine:** Implemented `core.py` to parse complex file structures, supporting custom prefixes and inferring metadata directly from TIFF headers (Voxel Size Z/Y/X).
- **Scan Order Support:** Added dynamic coordinate mapping for Raster, Row-Serpentine, and Column-Serpentine (`pan-ASLM`) acquisitions.
- **Streamlit UI:** Created `app.py` as an interactive form to intake dimensions, overlap strategies, and execution targets (Misha Slurm vs Local).
- **Bundle Generator:** Engineered the output to generate `dataset_manifest.json`, `stitch_settings.txt`, and executable run scripts (`.sbatch`, `.sh`, `.bat`) with dynamic relative or absolute path resolution.

### Usability & Pre-visualization Upgrades

- **Tiles View & Benchmark:** Created symlink/hardlink logic (`generate_tiles_view`) to safely inspect dataset subsets.
- **Interactive Grid Preview:** Integrated a pixel-perfect 2D Matplotlib mini-map grid preview overlaid with acquisition order arrows, allowing users to visually verify their coordinate permutations before spending cluster resources.
- **Auto-Detect Backend:** Scripted intelligent discovery of the `pi2` conda environment and entrypoints, eliminating manual path configurations.

### Stage 1: Advanced QC - Warping Diagnostics

- **Deformation Analytics:** Built a robust parser (`parse_local_shift_files`) for `pi2`'s `world_to_local_shifts_*.raw` outputs.
- **Statistical Insights:** Implemented magnitude extraction, calculating 95th percentile limits, Max Displacements, and Component Drift (X, Y, Z translation bias).
- **Visualizations:**

- Integrated a log-scale Warping Magnitude Histogram.
- Embedded a 2D Spatial Heatmap (Hexbin density overlay) to intuitively highlight local deformation hotspots and identify problematic tile seams.
- **Reporting:** Added a `plot_warping.py` Python script to the output bundle, enabling exact reproduction of the graphs outside the GUI in a Nature-publication-ready 8pt, 180mm PDF format (`qc_warping_spatial.pdf`).

## Stage 2: Advanced QC - Intensity Drift Analysis

- **Sub-sampling & Intensity Metrics:** Created analytical loops within `core.py` to sample a 70% XY crop across 12 Z-planes (~200k voxels) per tile, reporting the 90th (`p90`), 50th (`p50`), and 10th (`p10`) intensity percentiles.
- **Robust Normalized Drift (S):** Implemented the metric  $S = \ln(p90) - \ln(p50)$ , effectively decoupling multiplicative hardware illumination drift from actual structural biology variations.
- **Trend Analysis:** Integrated `scipy.stats` (with grace-fallbacks to `numpy`) to calculate Theil-Sen robust slopes and Spearman  $\rho$  fit quality.
- **Saturation Control:** Tracked `sat_frac` (Saturated Voxel Percentage) based on physical hardware bit depth.
- **Visual Output:** Generated a 3-panel Plotly/Matplotlib visualization showing Unnormalized Peak Intensity, Robust Normalized Drift (+ rolling median), and Saturation.
- **Gain Correction Handoff:** Added logic to dynamically detect unacceptable signal drop-offs ( $>5\%$  &  $\rho < -0.3$ ) and automatically scaffold `generate_gain_corrected_stacks`.

## Recent Polish (Late February 2026)

- **Regional Edge Analysis:** Grouped warping data spatially (Top, Bottom, Left, Right) to detect directional drag (e.g., stage axis slipping).
- **Interactive Data Drilldown:** Integrated `plotly.express` for hoverable hexbin data visualization.
- **Output Standardization:** Pinned all analytical graphics to high-resolution vector PDF generation with associated JSON payload files (`qc_intensity_timeseries.json`) for data provenance.
- **Refined Code Architecture:** Stabilized Windows execution with hard UTF-8 charmap encodings and strict Streamlit indentation block management within custom `with st.expander` environments.

## Conclusion

The NRStitcher Preprocessing Pipeline is fully operational, bridging the gap between raw microscope outputs and highly complex pi2 stitching logic. The pipeline significantly lowers the barrier to entry for end-users, enforces dataset integrity before execution, and provides world-class quality control graphics to defend the integrity of the data post-stitching.